AI Incident Report Assistant M1 Submission

· Optimus Prime

1. Problem statement and our understanding

1.1 Understanding

- **Goal:** turn a natural spoken incident description into a structured, reviewed incident report, end to end on the device.
- **Design principle: the model proposes, the app decides.** A small on-device model will sometimes return malformed or unsupported output. Validation, the mandatory-field gate and submission are therefore deterministic app code, never the model.1.3 Target platform

Target platform

Item	Value
Device	QIDK, Snapdragon 8 Gen 3 (SM8650), Hexagon v75 NPU
OS and runtime	Android; QAIRT, with Genie for the LLM
Connectivity	Fully offline; no cloud APIs for any AI step

2. Idea / Abstract

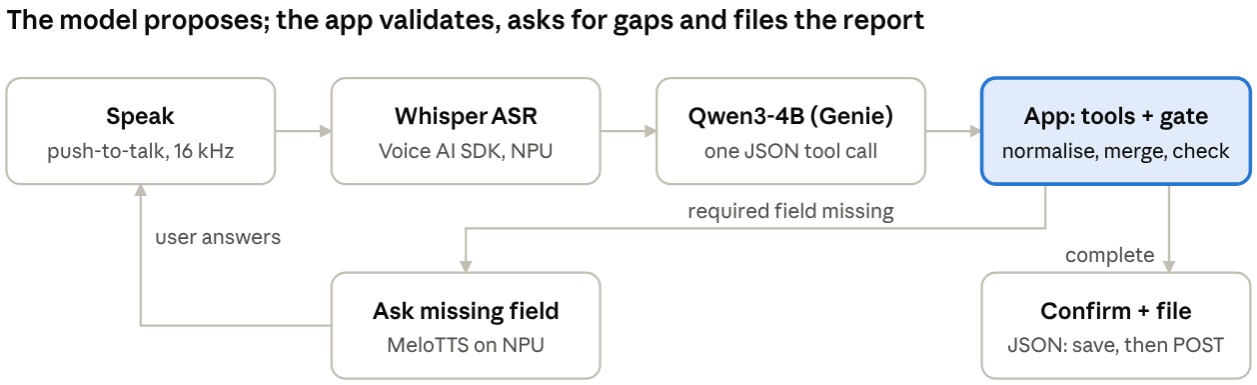
A one-tap voice assistant for incident reporting. Tapping a home-screen widget starts recording. The user describes the incident once, the form fills itself, and the assistant asks only for required fields that are still missing. It then reads back a summary and files the form when the user says yes. And if any mandatory fields are missing it asks the user to answer again.

The core design involves on-device Whisper which transcribes the speech. We will use LLM, running on the NPU through Qualcomm's Genie runtime, only proposes field values, as one JSON tool call per utterance. App code normalises, merges and validates every value, enforces the mandatory-field gate, chooses the next question, and submits only after explicit confirmation. MeloTTS speaks the questions and confirmations.

Everything is **schema-driven**. The form is a JSON definition: fields, required flags, types, options with aliases, questions and question groups. The prompt, tools, validation, UI and output are all generated from it, so a new form or a different model needs no code change. The output is one report JSON with values, confidence, evidence quotes, transcript and timings, ready to save or post to any API, and also ready to update form API.

3. High-level approach, pipeline and architecture

Each utterance makes one pass through the pipeline; turns repeat until every required field is filled and the user confirms.



one reporting turn · 3 models on the NPU, decisions in app code

4. Planned testing methodology

Each judging criterion gets its own metric, measured first on the host and then on the QIDK. Targets below are our own provisional goals; the deck sets none.

- **Host tests (no device):** unit tests for the normaliser, parser repair, merge policy, gate and fast intents. The water-leak scenario runs end to end with scripted model outputs. Property tests check that random field updates never break the gate.
- **Test set:** a 20-utterance core set, extended to about 60 scripted scenarios. They cover incident types, incomplete reports, corrections, negation ("nobody was hurt") and two incidents in one utterance. Each is recorded by two speakers, in a quiet and a noisy room. Gold field values are written before recording. Prompt tuning uses a development split; a held-out split is run once for the final numbers.
- **Baselines:** the same LLM on reference transcripts, which separates ASR errors from extraction errors, and the rule-based EchoLlm, which shows what the LLM adds.